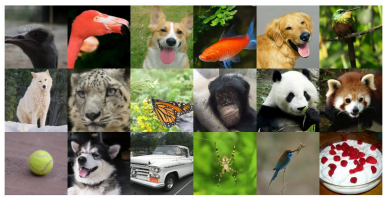


Diffusion Models

From noise to images: how modern generators work

- Dans ce dernier cours, on va parler d'une famille de modèles dédiés à la génération, notamment la génération d'images : les **modèles de diffusion**.
- Notre objectif va être de comprendre la recette qui se trouve au cœur de ces modèles et qui consiste, en deux mots, à transformer du bruit en images.
- On va commencer par une vue d’ensemble de la génération, puis se focaliser sur les mécanismes précis de ces modèles, comment on entraîne ces modèles, comment on les adapte et comment on les accélère.
- Il y aura un peu de mathématiques, mais seulement deux formules centrales à retenir que je signalerai explicitement
- Du coup, n’hésitez pas à m’arrêter si des choses ne sont pas claires
- **Transition :** « Commençons par quelques exemples de ce que cette recette permet de faire. »

Why diffusion models?



Class-conditional generation
Dhariwal & Nichol, 2021



Text → image
Stable Diffusion (Rombach et al., 2022)



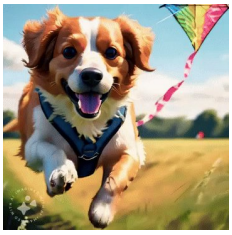
Outpainting
DALL·E 2 — August Kamp × OpenAI, 2022

- Major family of image and video generators
- Generate: from a class label, from a text prompt, from an existing image
- The model synthesizes an output from what it learned

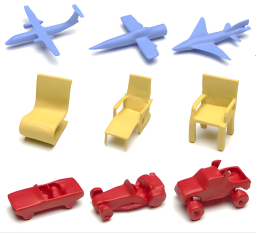
Same recipe behind all three: from pure noise to a new image

- Voici trois exemples de génération d'images par diffusion. À gauche, on part d'une classe, par exemple une catégorie d'objet ou d'animal. Au centre, on donne un texte, un prompt, et le modèle produit une image correspondant à une description.
- À droite, on part d'une image existante et on demande au modèle de l'étendre au-delà de ses bords : on appelle ça l'outpainting: on demande en effet pas toujours de créer de zéro aux modèles de diffusions.
- Dans les trois cas, l'objectif est de produire un résultat plausible et d'obtenir de la variété, à partir de ce que le modèle a appris sur son jeu d'entraînement. L'idée est de lui faire apprendre à généraliser.
- Le point commun entre ces méthodes, il est annoncé en bas de la slide : on part d'un bruit pur et on le transforme progressivement. On verra plus loin comment une condition (ici une classe, un texte ou une image à compléter) intervient dans cette transformation.
- **Transition :**« J'ai parlé d'images ici, mais la même idée fonctionne aussi sur d'autres types de données. »

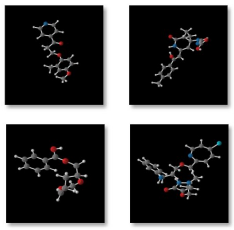
Beyond images: video, 3D, molecules, science



Video — Emu Video (Meta)



3D shapes — LION



Molecules — GeoLDM

...also in Science:

AlphaFold 3: structure

RFdiffusion: proteins

GenCast: weather

- Same recipe, different data:
  - video = images + time
  - shape = 3D coordinates
  - molecule = atom types + 3D coordinates
- Representations and noising rules depend on the data
- Goal: samples that are **realistic AND diverse,** not one averaged answer

Sources: ai.meta.com/blog/emu-text-to-video-generation-image-editing-research · arxiv.org/pdf/2210.06978 (LION) · arxiv.org/pdf/2305.01140 (GeoLDM)

- La **vidéo**, par exemple, qui ajoute une dimension temporelle : il ne suffit pas de produire de belles images isolées, il faut aussi garder une cohérence entre les images successives.

- On peut également générer des objets 3D ou travailler sur des molécules et des protéines. On manipule plus des pixels, mais des coordonnées 3D et des types d'atomes.

- Dans l'encadré de droite, je mentionne des applications scientifiques. En météo, par exemple, produire plusieurs futurs possibles ça permet d'explorer une incertitude, au lieu de ne fournir qu'une seule prévision moyenne.

- L'idée de la diffusion reste la même, mais on doit adapter le réseau de neurones sous-jacents, et la manière de gérer les données.

- **Transition :** « Dans ce cours, on va se concentrer sur la génération d'images. Du coup, commençons par un petit aparté sur ce qu'est une image. »

What is an image ?

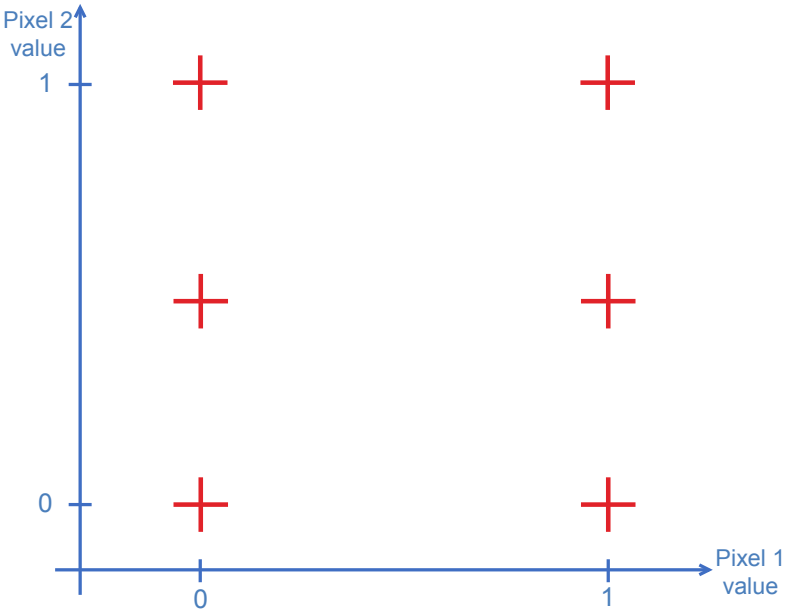


$$\overbrace{N_{width} \times N_{height}}^{N_{pixels}} \times N_{channels} = \text{total number of values that defines an image (big number)}$$

→ **Point (or vector) in a (very) high dimensional space**

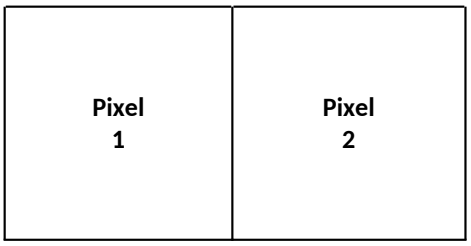
- Voici une image du cluster Jean Zay, dont je vais me servir plusieurs fois dans le cours
- Vous l'avez vu avec Thibaut hier, une image est constitué de pixels, et chaque pixel contient parfois plusieurs canaux, qui encodent la couleur de l'image sous forme de chiffres.
- Du coup, on peut représenter une image comme tous ces chiffres mis bout à bout, une longue (voire très longue) liste de valeurs, plusieurs millions pour une photo comme celle-ci.
- Mathématiquement, on peut interpréter cette liste de chiffres comme les coordonnées d'un point dans un espace avec beaucoup beaucoup de dimensions
- **Transition :** C'est important pour le reste du cours, donc je vais illustrer cette idée avec un exemple très simple

What is an image ? Simple case



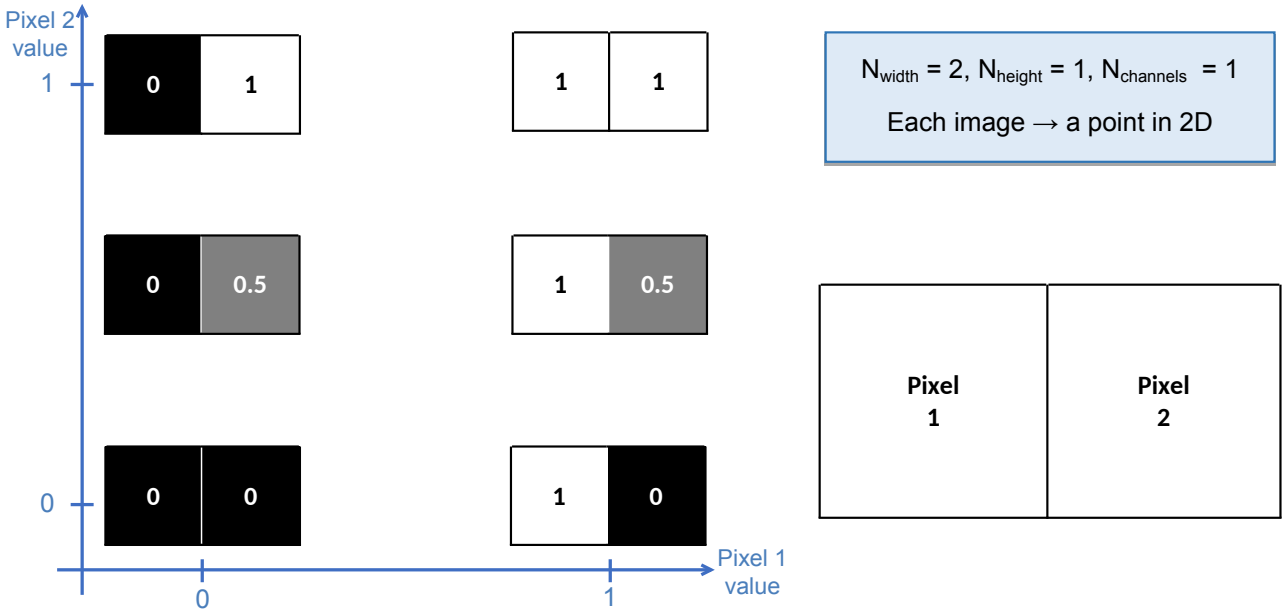
$N_{width} = 2$, $N_{height} = 1$, $N_{channels} = 1$

Each image → a point in 2D



- Prenons un exemple d'image très très simple : juste 2 pixels de largeur, 1 de hauteur, et 1 seul canal (en niveau de gris quoi)
- On a besoin que de deux valeurs pour décrire une telle image : val pixel 1, val pixel 2,
- Donc je peux représenter n'importe quelle image par un point dans un espace à seulement 2 dimensions, en fonction de la valeur du 1er pixel (abscisse) et la valeur du 2eme pixel (ordonnées)
- **Transition :** Ici j'ai choisi six points dans cette espace...

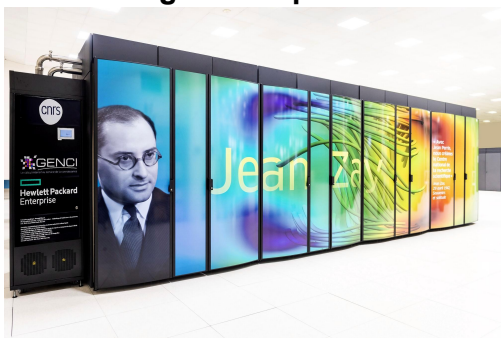
What is an image ? Simple case

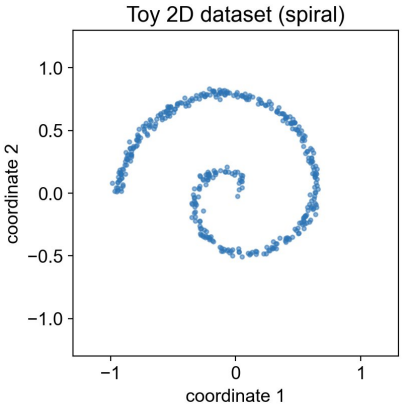


- ... Qui correspondent donc à ces six exemples d'images possibles, mais il y en a une infinité

A dataset is a cloud of points

One image = one point/vector





All pictures of the Jean Zay cluster “live” in a particular structure, a “cloud” in a very high-dimensional space

- Maintenant : si on s’intéresse à une famille d’image (p.ex, toutes les images possibles réalistes de chat, d’arbres, ou ici du cluster Jean Zay)
- À chaque image possible correspond donc un point dans un espace à très haute dimension
- La chose clé à visualiser, c’est que toutes ces images possibles (de Jean Zay par ex) forment une structure particulière, un nuage de points spécifique dans un espace à très haute dimension
- Dans ce cours j’ai choisi d’illustrer ça de manière abstraite avec un nuage en forme de spirale, limité à deux dimensions ; bien entendu dans la vraie vie c’est beaucoup plus complexe
- À chaque famille d’image correspond un nuage de points différents
- **Transition :** Si on reformule ça dans un langage un langage un peu plus mathématique...

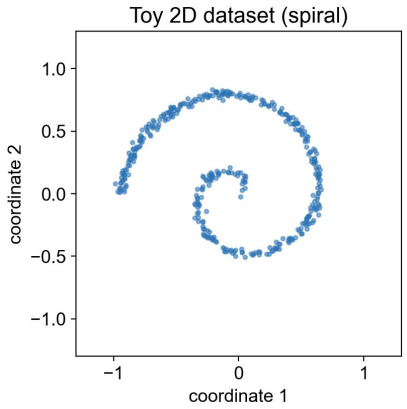
A dataset is a cloud of points

One image = one point/vector = x

q(**x**): density of real data
(q(**x**)~1 means **x** resemble the object of interest)

Goal: generate samples with the same statistical structure

But: we observe examples, not the distribution



Toy 2D distribution — one point per sample

Important note: random point (almost) always look like noise

- Je vais utiliser la notation X dans la suite pour définir une image / un point dans cet espace
- Pour une famille d'image donnée (par ex cluster Jean Zay) on peut représenter mathématiquement le nuage de points correspondants par une densité de probabilité q(X)
- Qu'est-ce que veut dire cette probabilité ?
* image X qui ressemble au pixel près à une vraie photo de Jean Zay : q~1
* image X qui n'est que du bruit blanc ou mélange absurde de formes : q~0
- Le but du jeu c'est d'être capable de générer des points de cette distribution
- Le souci : on a pas la formule de cette distribution, on connaît juste des exemples
- Si on prend un point au hasard dans l'espace, quasi sûr que ça ressemble à du bruit pur
- **Transition :** « Par où on commence pour faire cette génération ? Comparons cette tâche à une tâche supervisée qui nous est familière : la classification d'images. »

Classification vs generation

Classification	(Unconditional) generation
Training data: pairs (x, y)	**Training data:** images **x**, without class labels
Model: $p_\varphi(y \mid x) \approx q(y \mid x)$	**Model:** $p_\theta(x) \approx q(x)$
Loss: cross-entropy vs a one-hot target	**Loss:** ? what would the target even be?
Output: class distribution → argmax	**Output:** a **new sample**, not a label

Note: p_θ = learned model, θ = learned parameters

Predicting an average image is not the same as sampling:
we want to model the range of possible answers, not just one average
➡ that's where diffusion excels

- À gauche, la **classification**:

* les données d'entraînement, c'est des paires image–label

* pour une image donnée, le modèle prédit une distribution de probabilité sur des classes possibles

* notre cible, c'est de minimiser une loss du type cross-entropy (en gros un écart entre deux distributions de proba)

*on choisit ensuite (en général) la classe prédite la plus probable.

- À droite, la **génération** (dans le cas sans condition, pas de prompts ou quoi que ce soit):

* les données d'entraînement, c'est juste des images

* l'objectif, c'est d'apprendre la distribution q(x) grâce à un modèle p_\theta

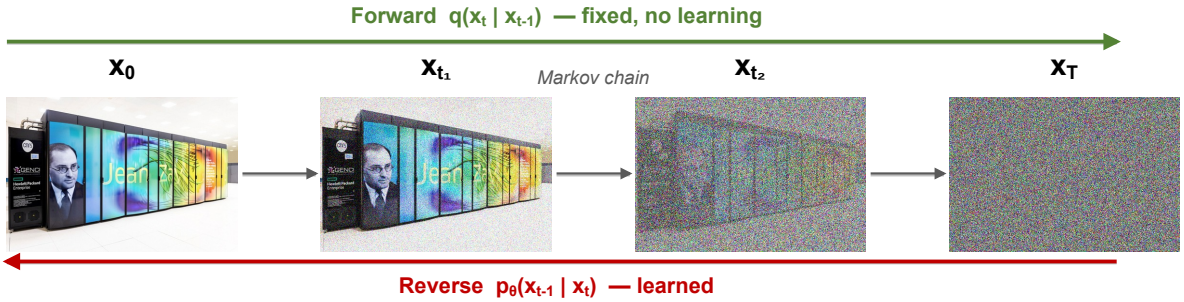
* quelle est la bonne cible/loss pour entraîner un générateur ? ...bonne question ! On n'a pas une unique « bonne image » à sortir pour chaque tirage aléatoire.

- En effet, on veut pas just prédire une image moyenne : on veut tirer une image parmi plein de réponses possibles. On veut une méthode qui puisse représenter la variété des images plausibles, pas seulement leur moyenne.

- Transition : « C'est en ça que la diffusion excèle. Regardons donc enfin en quoi consiste cette recette. »

Diffusion = two processes

Here: focus on Denoising Diffusion Probabilistic Models (DDPM, Ho et al. 2020)



Forward (fixed): add a little noise, T times, nothing to learn

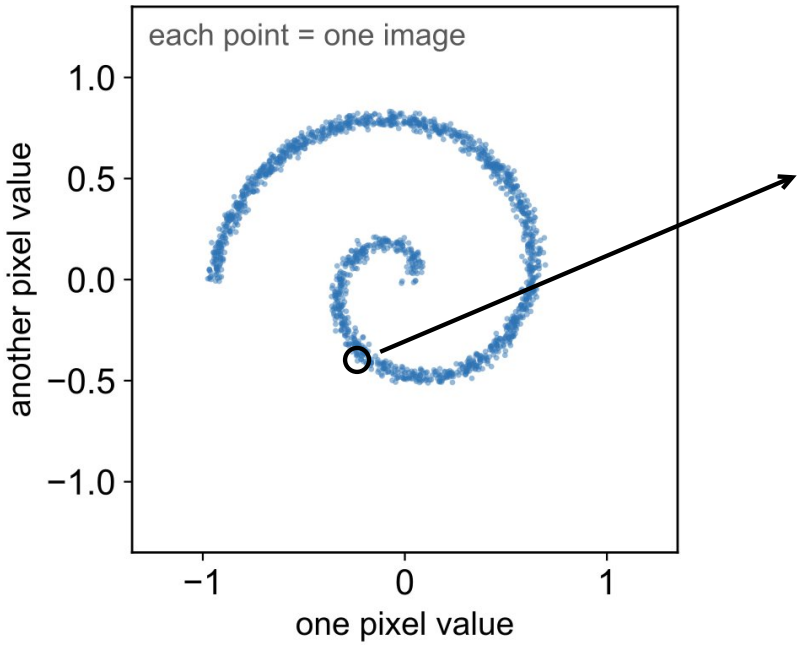
Reverse model: learn transitions back toward plausible data (not the exact steps)

Markov chain: each step only depends on the previous one

DDPM: Ho et al., 2020 · idea from non-equilibrium thermodynamics: Sohl-Dickstein et al., 2015

- Dans ce slide, on a les deux processus au coeur du modèle de diffusion “historique” qu’on va utiliser comme référence ici, à savoir le DDPM, pour “modèles probabilistes de diffusion par débruitage” proposé par Ho et collaborateurs en 2020
- La flèche du haut représente le **processus direct** : on part d'une donnée, ici l'image de Jean Zay, et on la corrompt progressivement, en plein de petites étapes, des centaines. Ce processus est fixé à l'avance : il n'y a pas de réseau à entraîner pour ajouter ce bruit.
- Pour les amateurs de statistiques : le processus direct est une chaîne de Markov, chaque nouvel état est construit à partir de l'état précédent.
- La flèche du bas représente le **processus génératif inverse** : on part d'un état très bruité et on apprend à revenir progressivement vers des données plausibles. C’est ces transitions que le modèle va essayer de représenter. Attention: on veut pas retrouver le film exact du bruitage d'une image particulière : on cherche à produire statistiquement des nouvelles images plausibles
- **Transition :** « On peut illustrer visuellement ces deux processus grâce à notre visualisation en spirale... »

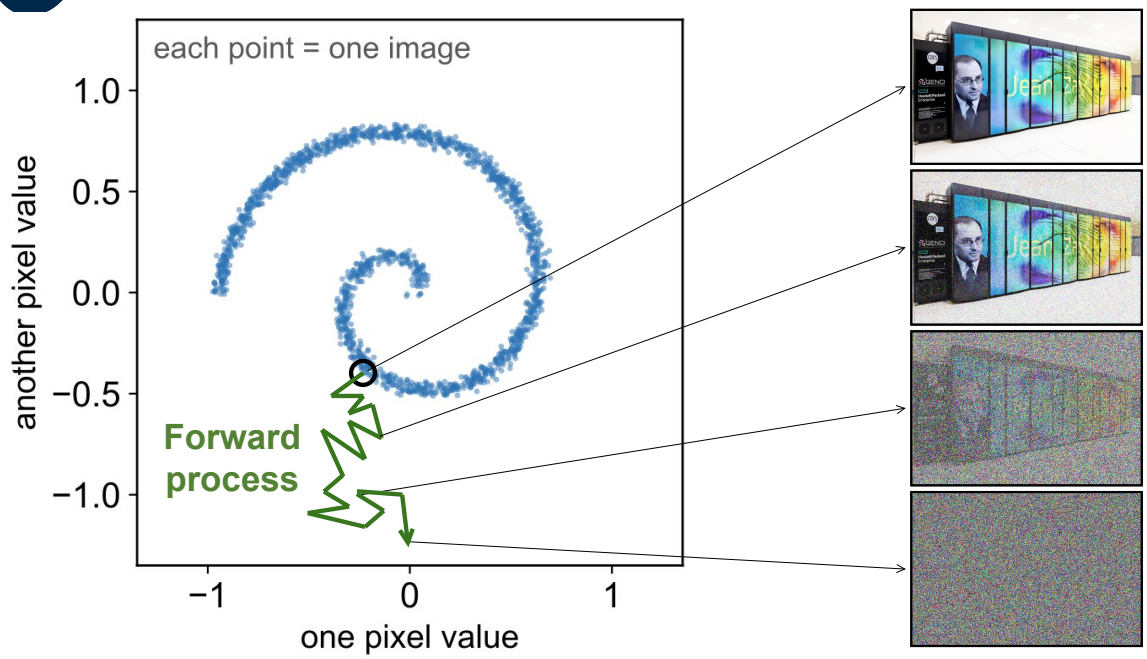
Diffusion = two processes





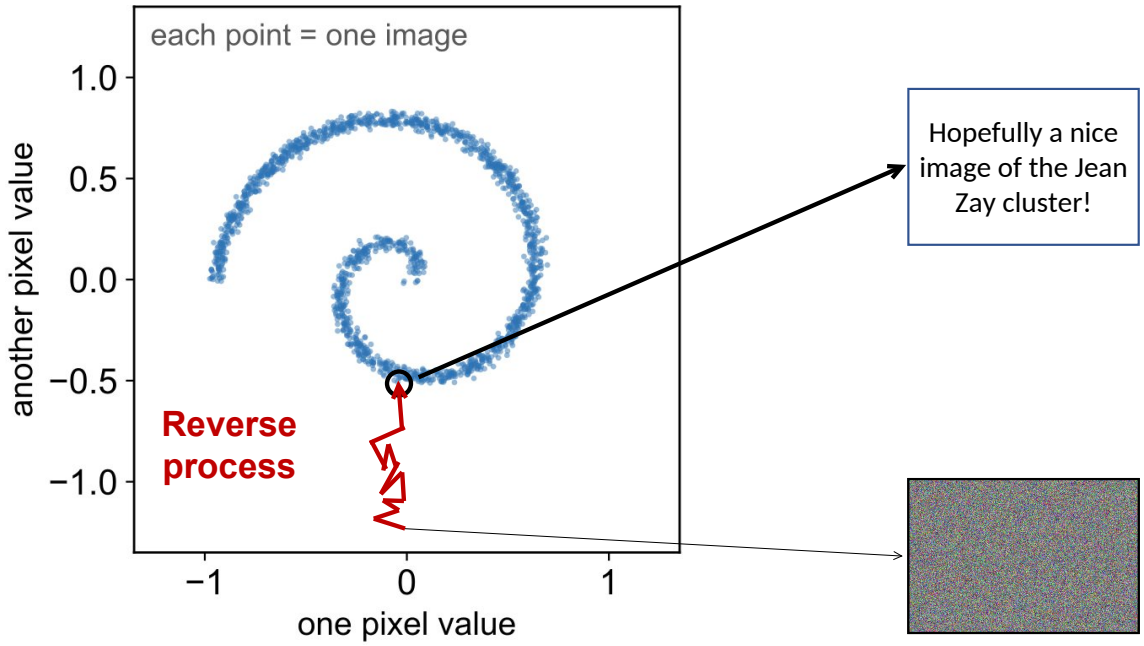
- La spirale, pour rappel, ça représente toutes les images possibles du cluster Jean Zay
- Le point de départ, c'est cette image-là particulière, représentée par un point particulier

Diffusion = two processes



- Dans le **processus direct** : on rajoute du bruit petit à petit, ce qui correspond à faire des sauts aléatoires dans cet espace
- Plus on s'éloigne de la spirale, plus on est bruité
- Les plus perspicaces d'entre vous auront remarqué que ça ressemble à la trajectoire de diffusion d'une particule, et c'est effectivement de la que la "diffusion" en machine learning prend son nom

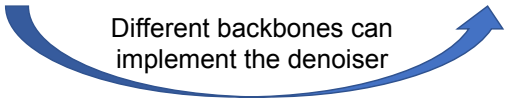
Diffusion = two processes



- Dans le **processus génératif inverse** : on part d'un état très bruité et on apprend à faire “les bons sauts” pour revenir progressivement vers des images de Jean Zay plausibles ; on ne revient pas forcément au point de départ, on veut juste revenir vers des images plausibles
- [Questions ?]
- **Transition**: « Avant de rentrer dans le détails de ces deux processus, je veux insister sur un point »

Diffusion = a recipe, not an architecture

Architectures (what the network is)	Diffusion (how we train and use it)
• Convolutional network (CNN) • Transformer • Graph neural network (GNN) → these are building blocks	**Training:** learn from corrupted data **Sampling:** iteratively transform noise into data Repeat T times → a new sample

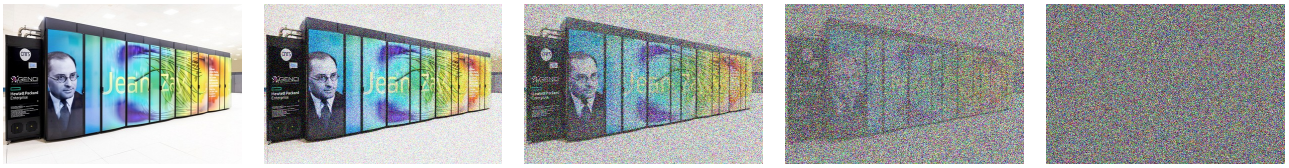


“Diffusion model” = the framework (corruption + objective + sampler), the network inside is just the denoiser

- Le point, c’est que la diffusion, c’est une recette, pas une architecture
- Les CNNs, les Transformers, les GNN : ça ce sont des **architectures**, des manières différentes d'organiser des neurones
- La diffusion : c’est plutôt une **recette d'entraînement et de génération**. C’est l’idée de construire des exemples corrompus, entraîner un réseau à faire une prédiction utile pour enlever ces corruptions, puis utiliser ces prédictions dans une procédure de génération.
- Le réseau qui fait des prédictions à l'intérieur de la recette peut changer : dans ce cours, on présente un U-Net, mais on peut aussi utiliser d'autres architectures.
- **Transition :** « Regardons maintenant les deux processus qui organisent cette recette, en commençant par le sens facile: ajouter du bruit. »

Forward: destroy the image

$x_0 \rightarrow x_1 \rightarrow \ldots \rightarrow x_t \rightarrow \ldots \rightarrow x_T$



t = 0 | t = 100 | t = 250 | t = 500 | t = 1000

- Add a bit more noise at every step, **T = 1000 is a hyperparameter** (historical DDPM choice)
- t = noise level
- **Fixed process: no parameter to learn**, no network involved
- At the final noise level, the distribution is close to N(0, I)

Vignettes recomputed with the real linear schedule (β_1 = 1e-4 → β_T = 0.02) · photo: Jean Zay, IDRIS/GENCI

- De gauche à droite, on observe la photo de Jean Zay à des niveaux de bruit croissants : l'image propre au départ, puis des états intermédiaires, jusqu'à un état qui ressemble visuellement à du bruit.
- À chaque étape, le signal est un peu atténué et du bruit est ajouté. Le processus est entièrement fixé par ce qu'on appelle le "schedule", le planning de bruit (on verra le détail un peu plus loin) : aucun paramètre de réseau n'est appris ici.
- Dans l'exemple historique DDPM, on utilise $T=1000$ étapes. C'est un choix de configuration, c'est pas une obligation pour tous les modèles de diffusion. La valeur de "t" vous donne le niveau de bruit de chaque image.
- À la fin, la **distribution** des images bruitées est proche d'une gaussienne standard. Spoiler : c'est cette propriété qui permet ensuite de démarrer la génération à partir d'un bruit gaussien.
- **Transition :** « Mathématiquement, comment on s'y prend pour le bruitage ? »

Forward: one step

$$q(x_t \mid x_{t-1}) = \mathcal{N}(x_t; \sqrt{1-\beta_t}\, x_{t-1}, \beta_t I)$$

$$x_t = \sqrt{1-\beta_t}\, x_{t-1} + \sqrt{\beta_t}\, z, \qquad z \sim \mathcal{N}(0, I)$$

Signal

$\sqrt{(1-\beta_t)}$ × previous state x_{t-1}

Fresh noise

$\sqrt{\beta_t}$ × new Gaussian noise z

Shrinking the signal keeps the overall scale controlled

- Prenons une seule étape du processus direct.
- En haut, la première ligne décrit la distribution de l'image "x_t" (niveau de bruit "t") connaissant l'image "x_{t-1}", donc avec niveau de bruit "t-1" juste en dessous
- La deuxième ligne, c'est la même chose, mais ça montre comment tirer concrètement cet image bruité, c'est la somme de deux termes:
- L'état précédent "x_{t-1}", multiplié par un facteur \sqrt{1-\beta_t} : le signal est légèrement atténué.
- On y ajoute un bruit gaussien, fraichement tiré aléatoirement à chaque étape, qu'on multiplie par \sqrt{\beta_t}.
- Le paramètre \beta_t fixe la **variance du bruit ajouté à cette étape (qui est reliée à son amplitude moyenne)**. On atténue le signal pour éviter que les valeurs de l'image augmente.
- **Transition :** « Les plus statisticiens d'entre vous se seront peut-être demandé : quand on entraîne le réseau, a-t-on vraiment besoin de simuler ces mille étapes de bruitage une par une ? »

Shortcut: jump to any t

$$\alpha_t = 1 - \beta_t, \qquad \bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$$

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I)$$

- Independent Gaussian noise terms combine into one Gaussian
- Signal coefficient and noise standard deviation vary with t
- One operation creates a training example at any noise level

★ **Formula 1 of 2: add noise**

- Il se trouve en effet qu'on peut résumer toutes les mini-étapes de bruitage en une seule étape !

- Si j'introduit \alpha_t comme 1-\beta_t, et \bar\alpha_t comme le produit des α de 1 jusqu'au jusqu'au niveau t.

- Voici alors la formule centrale qui permet de fabriquer directement l'image bruité x_t à partir de l'image propre x_0

- Le premier terme correspond au signal, pondéré par $\sqrt{\bar\alpha_t}$; le second est un bruit gaussien pondéré par $\sqrt{1-\bar\alpha_t}$.

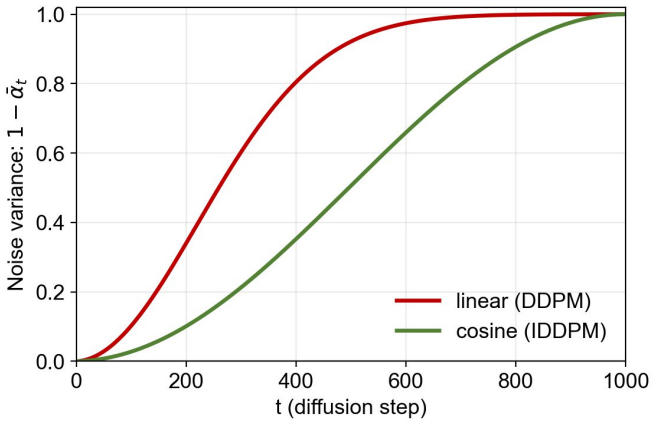
- Pourquoi ce raccourci est-il possible ? Plusieurs gaussiennes indépendantes qui s'ajoutent peuvent être regroupées en un seul bruit également gaussien. On note ce bruit cumulé \epsilon.

- À faible niveau de bruit, alpha est proche de un, le bruit est négligeable ; à bruit très élevé, alpha devient proche de 0, et le bruit domine.

- C'est la **première formule à retenir** : elle permet de créer en une opération un exemple bruité pour n'importe quel niveau de bruit choisi pendant l'entraînement.

- **Transition:** « Dernière étape : il reste à choisir comment le niveau de bruit évolue au cours des étapes. »

Noise schedule: linear vs cosine

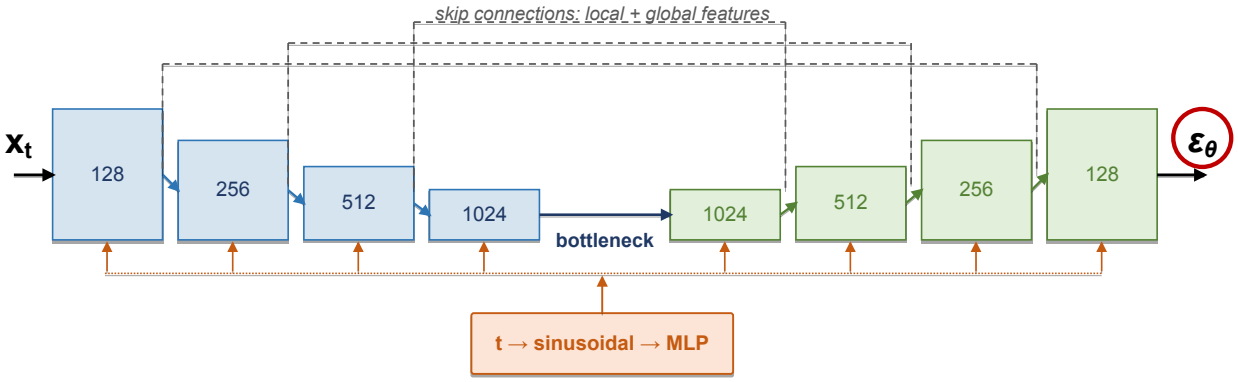


- β_t sets **how fast** the image is destroyed
- DDPM linear: $\beta_1 = 10^{-4} \rightarrow \beta_T = 0.02$
- Here, the linear schedule removes signal quickly
- Cosine preserves useful signal for more of the trajectory

- **The schedule controls the noisy inputs seen at each t**
- **The vertical axis shows the noise variance**

- Comme on a dit, c’est ce qu’on appelle le “schedule”, le planning du bruit, la manière dont on choisit les quantités de bruit au fil des étapes.
- En rouge, c’est le schedule dit “linéaire” de DDPM : \beta_t augmente de 10^{-4} à environ 0.02.
- Sur le graphique, l'axe vertical ne représente pas directement β_t en fonction du niveau de bruit, mais $1-\bar\alpha_t$, la variance du terme de bruit dans la formule directe que nous venons de voir (donc le carré de l’amplitude du bruit)
- Dans le schedule linéaire atteint rapidement une variance proche de un : le signal devient donc très faible assez tôt. Le schedule cosinus en vert est une alternative, il conserve du signal sur une plus grande partie de la trajectoire. Ce sont deux choix particuliers de répartition des niveaux de difficulté, pas un classement universel.
- **Transition :** « On a maintenant fini avec le bruitage et sa définition. Voyons maintenant le processus inverse, comment le réseau essaie de prédire le bruit et l’enlever. »

The denoiser: U-Net + time embedding



- t → sinusoidal embedding → MLP → injected into residual blocks
- Same resolution in and out ; skip connections combine local and global features
- Numbers: example channel counts ; alternatives include Transformers

Schematic redrawn for this course · U-Net: Ronneberger et al., 2015 · DDPM: Ho et al., 2020

- Regardons d'abord le réseau qui fait cette prédiction de bruit, dans le cadre du DDPM

- Il s'agit d'un **U-Net**, illustré dans le schéma ici. Vous avez déjà vu brièvement ce réseau convolutionel avec Thibaut hier

- Ce qui diffère, c'est que en bas, l'information du niveau de bruit "t" est est injectée dans les blocs du réseau, à travers un encodage, puis un petit réseau de type MLP. C'est ça qui permet au même U-Net de travailler à différents niveaux de bruit d'image.

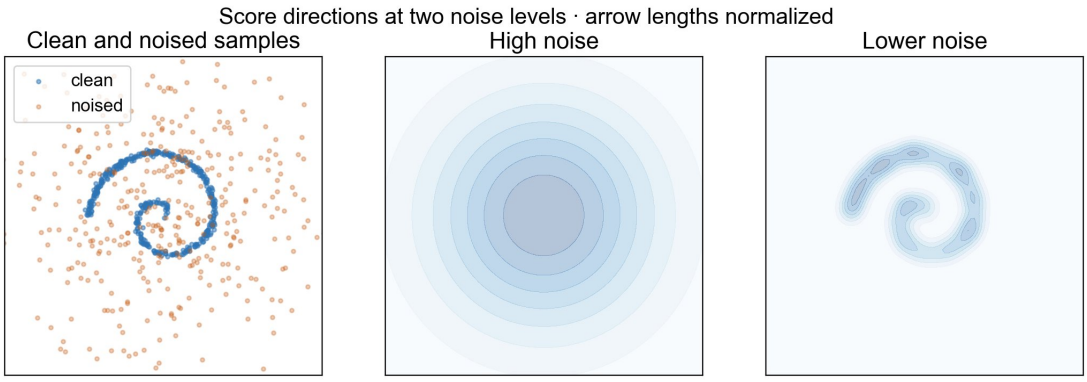
- L'entrée est une image bruitée ; la sortie est une **prédiction du bruit**, notée \epsilon_theta : c'est ce que le réseau pense être le bruit total qui a été ajouté à l'image originale

- Le réseau ne renvoie donc pas directement l'image débruité

- Comme dit avant, le U-Net n'est pas le seul choix possible : des Transformers sont aussi utilisés.

- **Transition :** « Est-ce qu'on peut se représenter visuellement ce que le réseau apprend? Oui! Toujours avec notre exemple de spirale. »

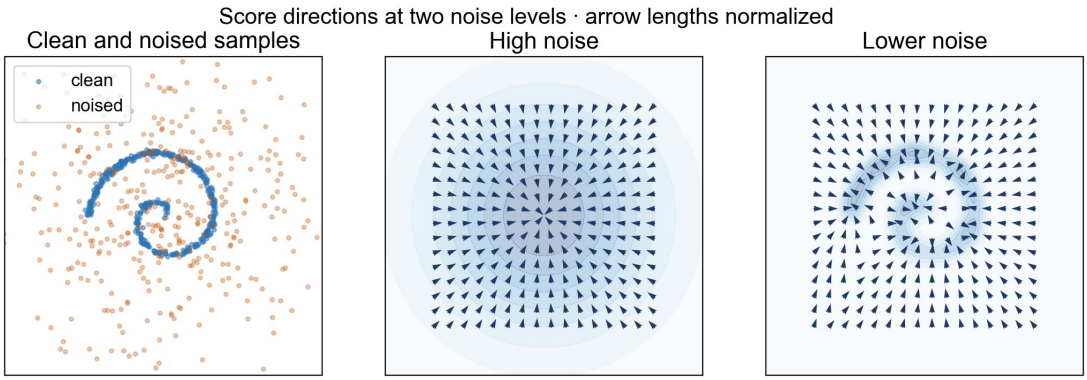
The 2D view: the network learns arrows



- À gauche, les points bleus représentent toujours notre distribution abstraite en forme de spirale, p ex toutes les images de jean zay (rappel : 1 point = 1 image) ; les points orangés montrent une version fortement bruitée. Bruiter les images (si vous vous rappelez) revient à déplacer les points bleus aléatoirement et du coup ça change la forme du nuage.

- Au centre et à droite, le fond bleu clair représente la densité de la **distribution de ces points après bruitage** pour deux niveaux de bruit, faible (on dévie un peu de la spirale) et fort (on a pour ainsi dire “oublié” la spirale).

The 2D view: the network learns arrows



$$s_\theta(x_t, t) := -\frac{\varepsilon_\theta(x_t, t)}{\sqrt{1-\bar{\alpha}_t}} \approx \nabla_{x_t} \log q_t(x_t)$$

The score points toward higher density in the noisy distribution

Exact empirical toy score · arrow lengths normalized to show direction

- En chaque point de la distribution, on peut tracer des flèches indiquent localement la direction vers une densité plus élevée : c'est ce qu'on appelle le **score** “s”.

- Quand on a beaucoup de bruit, la structure fine de la spirale est largement effacée et les directions sont grossières. À plus faible bruit, on commence à voir des directions qui reflètent davantage la structure de la spirale.

- Ces flèches illustrent précisément la quantité que le réseau cherche à approcher : pour un niveau de bruit donné, et une image (càd un point dans l’espace), c’est la “direction” moyenne où aller pour retrouver des images réalistes (=se rapprocher de la spirale). En effet, le score (ces flèches donc) peut être calculé en prenant la prédiction du réseau \epsilon_\theta, on change son signe et on applique un facteur qui dépend du niveau de bruit.

- **Transition:** « Voici donc ce que le réseau prédit. Maintenant, comment se passe l’entraînement des poids du réseau, concrètement ? »

Training: predict the noise you added

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{x_0,\, t,\, \varepsilon}\left[\|\varepsilon - \varepsilon_\theta(x_t, t)\|^2\right]$$

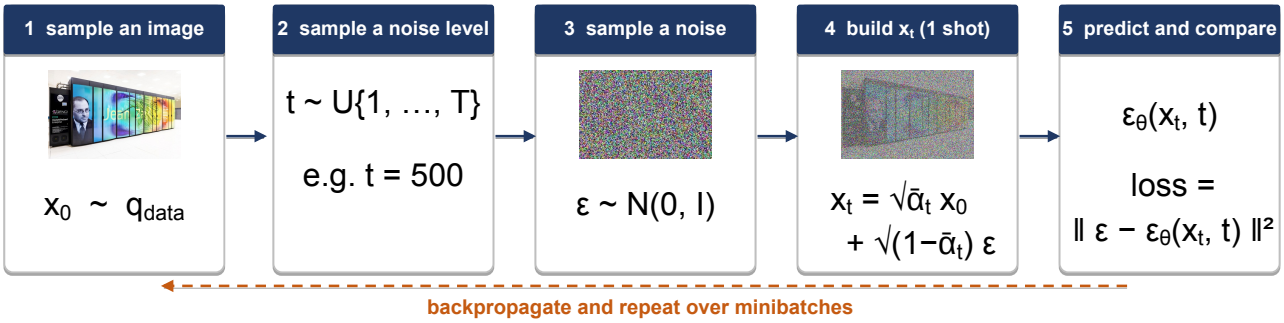
One training step:

- Sample image x_0, noise level t and Gaussian noise ε
- Construct x_t using the shortcut
- Compare network noise prediction ε_θ to real noise ε
- One network evaluation and backpropagation per batch
- No unrolling of the full diffusion trajectory

★ **Formula 2 of 2: learn to predict the noise you added**

- On va enfin répondre à la question de quelle loss on utilise ; voila comment se passe une étape d'entraînement
- On prend une image propre du jeu de données, on tire un niveau de bruit t et on tire un bruit gaussien \epsilon.
- Grâce à la première formule, on construit directement x_t.
- On donne ensuite (x_t,t) au réseau et on lui demande de prédire le bruit utilisé pour produire cet exemple.
- La formule affichée est la **deuxième formule centrale** : on compare, via une MSE loss, le bruit tiré et le bruit prédit, et on met à jour les poids.
- Une mise à jour d'entraînement ne déroule pas toute la chaîne de 1 000 pas : on construit directement un niveau de bruit choisi, puis on effectue une évaluation du réseau et une rétropropagation pour le batch.
- **Transition :** « On a tous les éléments de la chaine, voici donc un résumé de la recette en cinq étapes. »

Training, step by step



- Sample t independently per image ; training covers all noise levels
- Loss = MSE between the true noise and its prediction (pixels, channels, batch)
- Compute depends on model and data ; optionally keep an EMA

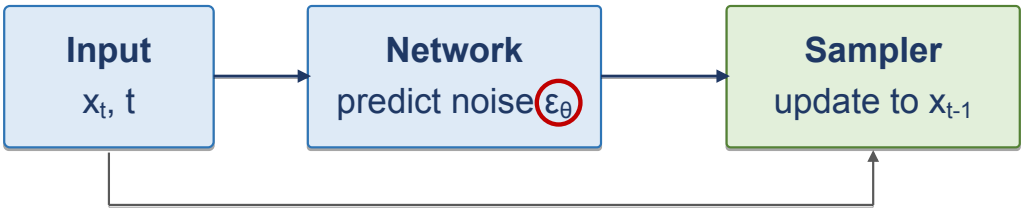
- De gauche à droite : on prend une image, on tire un niveau de bruit et un bruit gaussien, puis on construit directement l'image bruitée.

- Enfin, le réseau prédit le bruit et on compare cette prédiction au bruit que l'on vient de tirer. On rétropropage l'erreur et on répète l'opération sur d'autres minibatches.

- [Questions ?]

- **Transition :** « Une fois les poids appris, la procédure d'utilisation du réseau est différente. On va voir comment on utilise un réseau entraîné pour générer des images. »

One prediction, then one sampling update



- Network: predicts noise
- Sampler: uses x_t, t, prediction and optional fresh noise
- The same network is reused at every noise level

Repeat N times: prediction + sampling update

- Voici comment ça se passe

- Un réseau reçoit une image bruité “x_t” avec son niveau de bruit “t”. Comme on a dit précédemment, la sortie du réseau est une **prédiction du bruit**, notée \epsilon_\theta, c’est ce qu’il pense être le bruit qui a été ajouté à l’image originale x_0

- Il y a une deuxième boîte, le “sampler” (“échantilloneur”), qui combine la prédiction avec l’image actuelle x_t, le niveau de bruit t et du bruit fraichement généré.

- Il produit alors l'état suivant de l’image, x_t-1, que l'on renvoie dans la boucle au niveau de bruit suivant.

- On réutilise le même réseau (mêmes poids) à chaque fois: c'est l'entrée “t” qui lui indique le niveau de bruit auquel il travaille.

- **Transition :** « Comment on écrit ça mathématiquement ? »

From predicted noise to one generation step

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\varepsilon_\theta(x_t, t)\right)$$

$$x_{t-1} = \mu_\theta(x_t, t) + \sigma_t z$$

- Prediction sets the reverse mean ; DDPM adds fresh noise
- σ_t: fresh-noise standard deviation, specified by the sampler
- z ~ N(0,I)
- No fresh noise in the final step

Use mean + noise as the next step and repeat the process

- La première ligne donne la moyenne prédite \mu_\theta(x_t,t). Elle prend l'image actuelle "x_t" à l'étape "t", et utilise le bruit total estimé par le réseau, epsilon_theta, pondéré par un facteur.

- Qu'est-ce que ça représente cette moyenne ? => L'image moyenne à l'étape t-1, ce que le réseau pense être l'image la plus probable à l'étape juste avant.

- La deuxième ligne est la mise à jour du sampler (dans DDPM) : on part de cette moyenne et on ajoute une variation aléatoire $\sigma_t z$. Le paramètre \sigma_t est fixé et règle l'amplitude de ce bruit ajouté. On va voir un peu plus loin le rôle de ce bruit ajouté.

- En résumé, le réseau prédit le bruit, puis le sampler utilise cette prédiction et l'image courante pour produire l'état suivant, et ainsi de suite.

- **Transition :** « À quoi ressemble l'algorithme de génération d'image ? »

Sampling: the loop

Algorithm 2 · DDPM sampling

1. $x_T \sim N(0, I)$
2. for t = T, …, 1:
3. $\quad z \sim N(0, I)$ if t > 1, else z = 0
4. $\quad x_{t-1} = (x_t - \beta_t \, \varepsilon_\theta(x_t,t)/\sqrt{(1-\bar{\alpha}_t)})/\sqrt{\alpha_t} + \sigma_t \, z$
5. return x_0

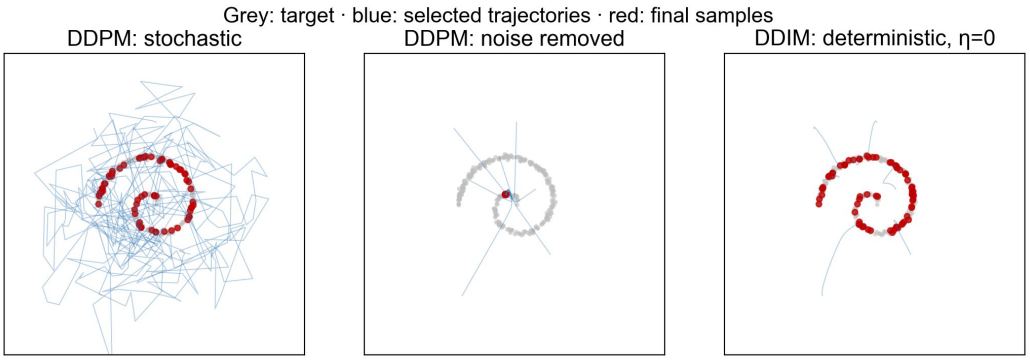
Sampler

- One network call per step
- Weights fixed at inference
- Fresh noise except final step

Training samples one noise level ; sampling visits decreasing levels

- L'algorithme DDPM commence par tirer x_T dans une gaussienne standard : c'est notre **point de départ aléatoire**. On parcourt ensuite les niveaux de bruit de T jusqu'à 1.
- À chaque étape de la boucle, le même réseau prédit le bruit à partir de l'état courant et du temps. Le sampler utilise cette prédiction pour calculer la moyenne, puis ajoute un bruit fraichement tiré ; au dernier pas, $z=0$.
- À la fin, l'algorithme renvoie x_0, l'échantillon généré, la nouvelle image générée
- Pendant cette boucle, les poids sont fixés : plus de rétropropagation
- Dans DDPM, il faut un appel réseau par étape : avec 1 000 étapes, cela fait 1 000 appels pour une image ou un batch d'images. C'est une source de coût que nous retrouverons plus loin.
- **Transition:** « Du coup, ce bruit/hasard ajouté pendant la génération, à quoi sert-il ? »

Sampling beyond the predicted mean



- DDPM samples around a mean ; removing its noise changes the distribution
- DDIM (Denoising Diffusion Implicit Models) uses a different deterministic update ; initial random noise still gives diversity

Calculated toy example · same starting points · 1,000 full updates

- Revenons une fois de plus à notre spirale

- Dans chacun des trois panneaux : les points gris sont les données cibles, en bleu ce sont des trajectoires de débruitage en partant des mêmes points initiaux, et les points rouges sont les échantillons finaux.

- À gauche, on applique l'algo DDPM avec son bruit ajouté à chaque étape. Au centre, on prend le même algo mais on retire le bruit aléatoire : on voit que les résultats se concentrent fortement et couvrent beaucoup moins la spirale. Le bruit est important pour la diversité des résultats.

- Voilà pour les bases de la diffusion ; dans le reste du cours, on va parler des raffinements qu'on peut apporter à cette recette de diffusion.

- À commencer par le panneau de droite : on utilise une recette de diffusion différente, DDIM. Elle est déterministe pour un point de départ donné (pas d'aléatoire), mais arrive quand même à produire des échantillons variés ! Donc le bruit au moment de générer des images est crucial pour la diversité pour DDPM, mais pas forcément crucial pour les modèles de diffusion en général.

- Transition : « Autre raffinement: et si je veux orienter ma génération d'images, avec par exemple une classe, un prompt, etc? »

Classifier guidance

$$s_\theta(x_t, t) := -\frac{\varepsilon_\theta(x_t, t)}{\sqrt{1-\bar{\alpha}_t}} \approx \nabla_{x_t} \log q_t(x_t)$$

$$s_{\text{guided}}(x_t, t, y) = s_\theta(x_t, t) + \lambda \nabla_{x_t} \log p_\phi(y \mid x_t, t)$$

- **Training:** fit a diffusion model **and** a classifier on noisy inputs
- **Sampling:** differentiate the target-class log-probability with respect to the sample
- Stronger guidance often improves class adherence at the cost of diversity

Dhariwal & Nichol, 2021 · Diffusion Models Beat GANs on Image Synthesis

- Une manière de faire ça, c'est de jouer sur le score qu'on a vu précédemment (je vous remets ici formule), ce qui revient à jouer sur la prédiction du réseau epsilon

- On a maintenant deux modèles : un modèle de diffusion et un **classifieur auxiliaire**. Ce dernier, on l'entraîne à évaluer la classe souhaitée, sur une image d'entrée bruitée, et le niveau de bruit "t".

- On écrit un nouveau score dit "guidé", qui additionne le score classique et gradient de la log-probabilité de la classe demandée. Intuitivement, ce gradient indique comment déplacer l'état courant pour le rendre plus compatible avec cette classe.

- Le coefficient \lambda règle la force du guidage. En pratique, un guidage plus fort peut améliorer le respect de la classe, mais souvent au prix d'une moindre diversité ou d'artefacts si l'on pousse trop loin.

- Cette méthode rajoute une coût : celui d'un second modèle et d'un calcul de gradient à chaque étape guidée.

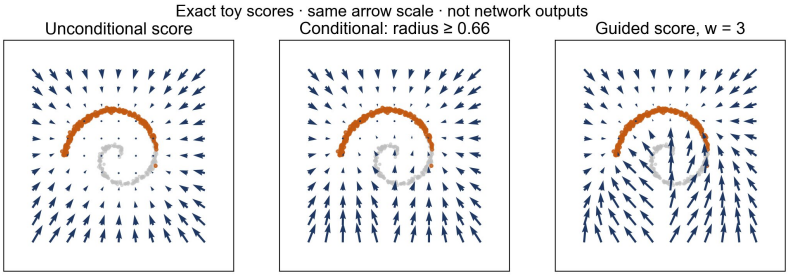
- **Transition :** « Peut-on obtenir un effet similaire sans entraîner ce deuxième classifieur ? »

Classifier-free guidance (CFG)

$$\hat{\varepsilon} = \varepsilon_\theta(x_t, t, \varnothing) + w[\varepsilon_\theta(x_t, t, y) - \varepsilon_\theta(x_t, t, \varnothing)]$$

conditional prediction

unconditional prediction



- w = 0: unconditional
- w = 1: conditional
- w > 1: amplified conditioning
- One network, trained with y or ∅

Two predictions per step ; useful strength depends on the model

- Oui ! Avec ce qu'on appelle le **classifier-free guidance** (ou CFG), qui joue non pas sur le score, mais sur la prédiction du réseau.

- On entraîne un même réseau de diffusion avec une condition (une classe d'image par exemple) et, une partie du temps, sans condition.

- À la génération, on calcule ainsi deux prédictions : une sans condition et une avec la condition. La différence des deux donne une indication de l'effet de la condition ; la formule montrée ici donne la prédiction finale, et permet de moduler l'effet de la condition avec le coefficient w.

- $w=0$ donne la prédiction sans condition, $w=1$ la prédiction conditionnelle, et $w>1$ amplifie la condition.

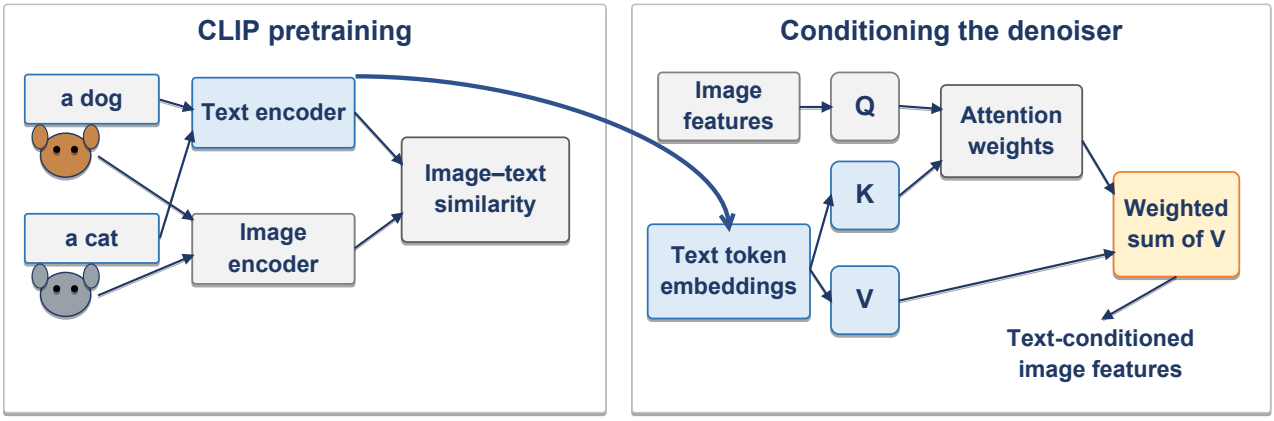
- Pour illustrer ça avec notre spirale, imaginez que la partie orange désigne la portion de spirale sélectionnée par la condition (par exemple, toutes les images du cluster Jean Zay, vue de dessus).

- Les trois panneaux comparent les trois régimes du coefficent w. Comme précédement, attention: un guidage trop fort peut réduire la diversité ou introduire des artefacts.

- Notez enfin le coût supplémentaire de devoir faire deux prédictions par étape (mais bon, toujours mieux que deux modèles à gérer)

- **Transition :** « Pour une classe, la condition est relativement simple. Maintenant, comment faire pour injecter p. ex. une phrase ? »

Text conditioning: CLIP + cross-attention

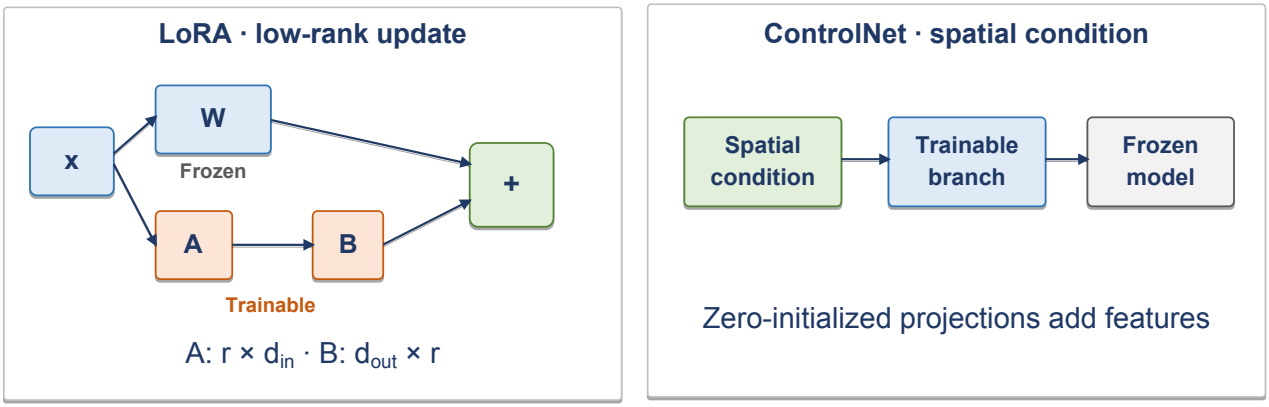


Q and K determine the weights; these weights combine V

CLIP: Radford et al., 2021 · drawings are schematic

- Le panneau de gauche illustre la methode de préentraînement de **CLIP** : un encodeur de texte et un encodeur d'image apprennent à rapprocher les représentations d'images et de descriptions qui vont ensemble.
- Ensuite, on réutilise l'encodeur de texte à droite dans notre débruiteur.
- On s'en sert pour créer une représentation abstraite d'un prompt donné; qui fournissent ensuite des clés K et V (si vous vous rappelez le cours de Martin hier).
- On compare K et Q, Q étant les caractéristiques l'image bruitée actuelle
- Les poids d'attention résultant sont combinés aux valeurs V, on obtient les caractéristiques de l'image enrichies par l'information du texte, et tout ça repart dans le débruiteur.
- **Transition :** « Une fois qu'on dispose d'un modèle préentraîné, on peut aussi l'adapter à un besoin précis. »

Adapting a pretrained model



Fine-tuning: monitor generations and overfitting, not only loss

- C’est ce qu’on appelle le fine-tuning, ça consiste à poursuivre l'apprentissage d'un modèle sur d'autres données.
- À gauche, dans la méthode **LoRA**, on garde la matrice de poids du réseau de départ W gelée et apprend une correction de faible rang, écrite comme le produit B x A de deux petites matrices A et B entraînables ; leur contribution est ajoutée à celle de W.
- À droite, la méthode **ControlNet** ajoute une branche entraînable, une copie exacte d’une partie du réseau initial (pas tout), pour apporter à l’image une condition spatiale : comme des contours, une carte de profondeur ou une certaine pose, par exemple. Les contributions de la branche sont injectées dans le modèle de base gelé.
- **Transition :** « En général, adapter un modèle peut coûter relativement peu. Mais la génération elle-même peut encore être lente. »

The sampling bottleneck

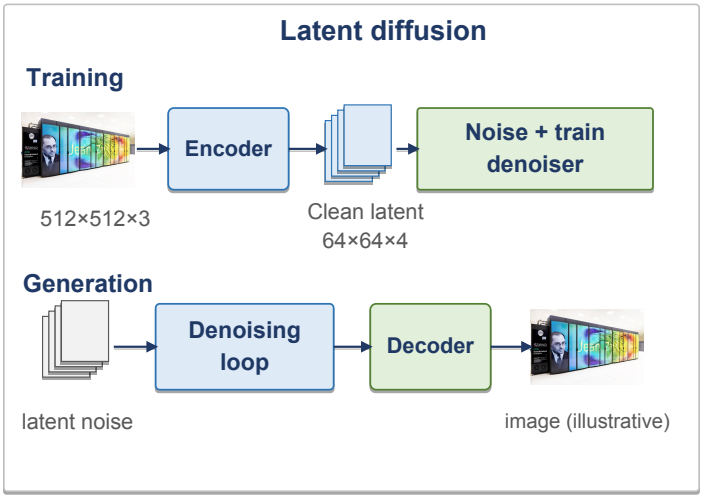
Family	Typical strengths	Common challenges
GAN	Fast sampling, high fidelity	Training stability, mode coverage
VAE	Fast sampling, useful latent space	Reconstruction / detail trade-offs
Diffusion	Sample quality and diversity	Repeated network evaluations

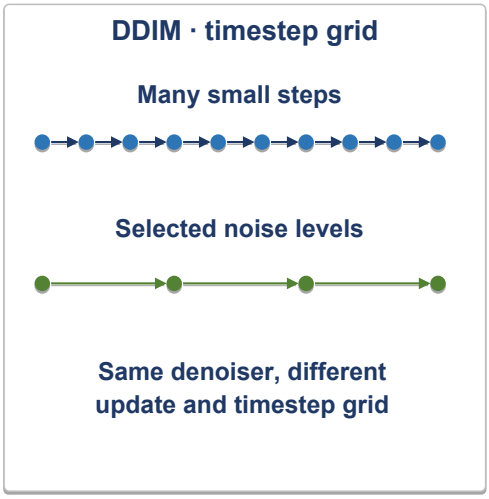
Historical overview, not a universal ranking

Two levers: cheaper evaluations & fewer evaluations

- Ce tableau est une comparaison historique des forces et faiblesses de diverses méthodes de génération d'image
- Pour les DDPM qu'on a présentés ici, la difficulté c'est vraiment le coût des évaluations répétées du réseau pendant la génération.
- On va regarder deux leviers : rendre chaque évaluation moins coûteuse, et effectuer moins d'évaluations.
- Transition : « Voici une approche pour chacun de ces deux leviers. »

Faster: latent diffusion + DDIM





- Dans le panneau de gauche, une méthode pour avoir des évaluations moins coûteuses : l'idée c'est de transformer, d'encoder les images en une représentation beaucoup plus petite, en perdant le moins d'info possible, dans ce qu'on appelle un espace latent. C'est dans cet espace qu'on ajoute le bruit et qu'on entraîne le débruiteur. C'est aussi là qu'on génère des nouveaux échantillons, qu'on les débruite, et à la fin on les décode pour reconstruire une image.
- À droite, une méthode pour avoir moins d'évaluations tout court : c'est le modèle DDIM qu'on a déjà mentionné. Si vous avez déjà entraîné votre débruiteur en mode DDPM, vous pouvez directement le réutiliser, c'est juste le sampler, l'échantilloneur qui change. On a une grille de temps plus grossière, on visite seulement certains niveaux de bruit, du coup la génération va plus vite.
- Transition : « Ces leviers, c'est rien de nouveau, mais ils restent intéressants avec des architectures et des objectifs plus récents. »

Beyond DDPM: backbones and objectives

Backbone	Objective	Data
U-Net or Transformer	Noise prediction or **flow matching**	Images, video, structures, weather

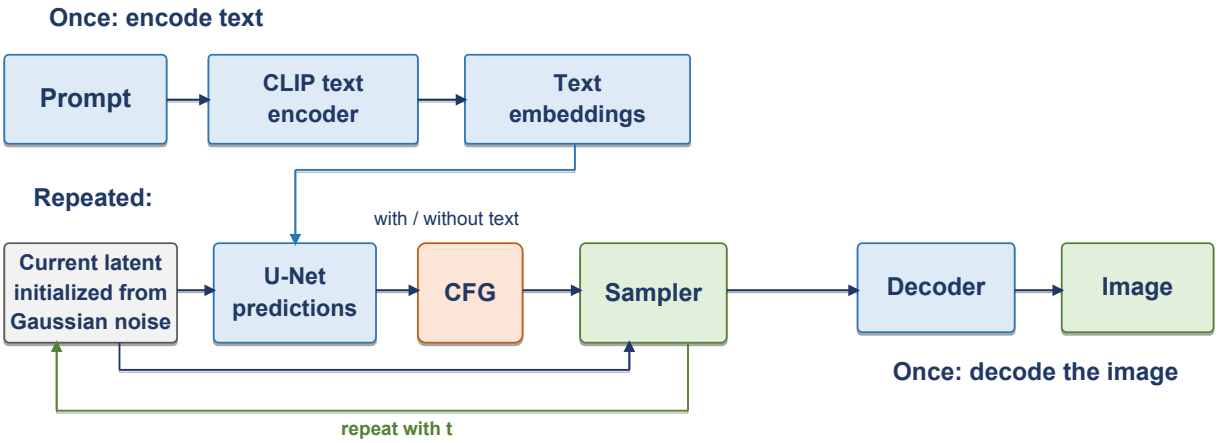
- Flow matching learns a velocity field along a chosen interpolation
- Training interpolations may be straight; generated trajectories need not be
- Learn to transform a simple distribution into data

- En parlant de modèle plus récents, de nos jours comme je vous l'avais mentionné, on utilise moins les U-Net et plus les transformers. Niveau données, on va bien au-délà des images : vidéo, structures diverses et variées...

- Et parmi les méthodes d'entrainement plus récentes, je voulais vous mentionner le *flow matching* : là où la diffusion classique fait faire à l'image un chemin sinueux et aléatoire à travers le bruit, Flow Matching force le modèle à apprendre une trajectoire en ligne droite entre le bruit et l'image réelle.

- **Transition :** « On a vu tous les briques nécessaires à un modèle de diffusion moderne; pour les assembler, prenons maintenant un modèle historique concret, le Stable Diffusion v1. »

Putting the pieces together: Stable Diffusion v1

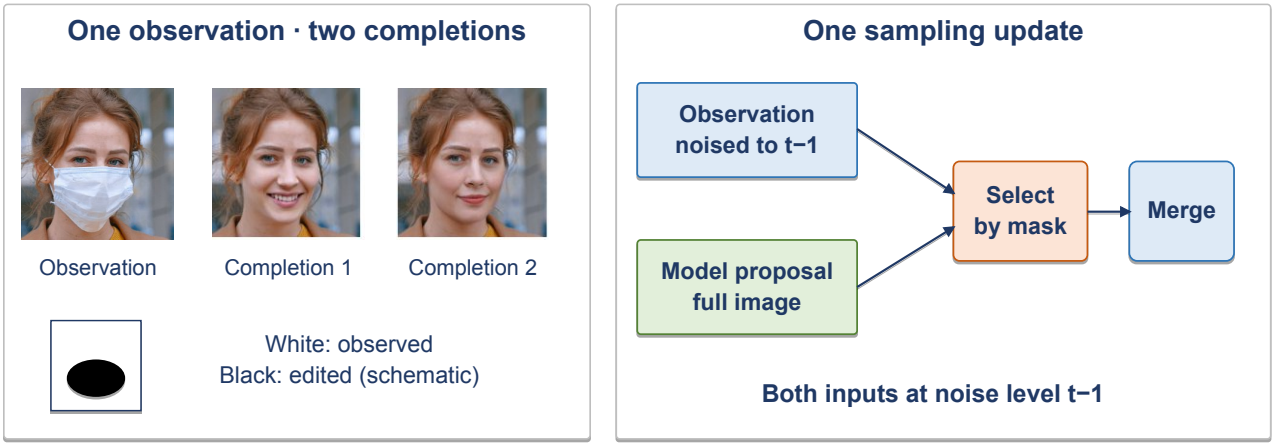


U-Net ≈ 860M · CLIP text encoder ≈ 123M · ~150,000 A100 GPU-hours

CompVis model card · SD v1-4 · LAION-2B-en and filtered subsets

- En haut du schéma, on encode une fois le **prompt** avec l'encodeur de texte CLIP. Les représentations qu'on obtient, elles sont réutilisées à chaque étape de la boucle.
- SD bosse dans l'espace latent :
- Au départ, l'image latente actuelle est initialisé à partir de bruit gaussien. À chaque pas, cette image latente, le niveau de bruit, et la condition textuelle sont utilisés par le U-Net pour produire les prédictions nécessaires.
- Avec CFG, on combine des prédictions avec et sans texte. Le sampler utilise cette prédiction guidée et image latente actuelle (liaison qui contourne le U-Net) pour produire l'image latente suivante. La boucle revient alors à la boîte « Current latent ».
- Une fois la boucle terminée, le décodeur transforme le latent en image.
- Je vous ai mis quelques ordres de grandeur : environ 860 millions de paramètres pour le U-Net, 123 millions pour l'encodeur de texte et un coût d'entraînement de l'ordre de 150 000 heures-GPU A100 (15 jours sur tout JZ, 2 jours sur H100)
- **Transition :** « Ce même type de modèle peut aussi servir pas seulement à générer, mais aussi à modifier une image existante. »

Editing with masks: RePaint



Same pretrained model · observed pixels enforced during sampling

Lugmayr et al., RePaint, CVPR 2022 · https://arxiv.org/abs/2201.09865

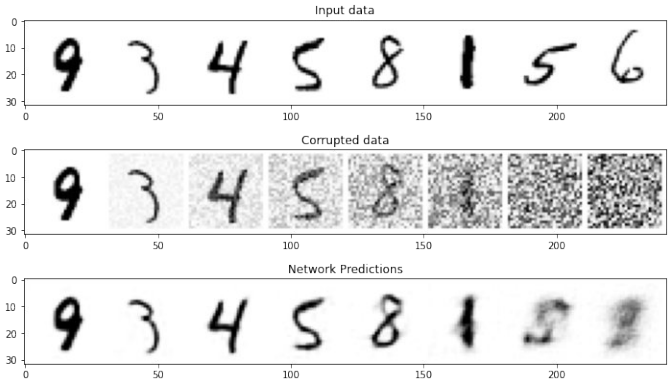
- À gauche, on voit une observation, avec une zone qu'on souhaiterait éditer. Et à côté, on a deux images édités possibles, plausibles, obtenues avec un modèles de diffusions.
- Dans la pratique, pour un pas allant de t vers $t-1$, le réseau et le sampler proposent un **état bruité complet**, et un masque ne conserve que la partie de cette proposition correspondant à la zone à générer.
- Pour la zone non masquée, on reprend l'image d'entrée en la bruitant au même niveau $t-1$, on fusionne avec la partie masquée générée, puis on continue la génération.
- **Transition :** « On arrive enfin à la conclusion de ce cours, avec un résumé des points à retenir. »

Take-aways & resources

1. Generation aims to reproduce a distribution and its diversity
2. Training: corrupt data and learn to predict added noise
3. Sampling: combine predictions with a compatible update rule
4. Random initial noise allows diversity with deterministic sampling
5. Conditioning steers; latent spaces and fast samplers reduce cost

- Il faut retenir d'abord que générer, ça consiste à apprendre à produire des échantillons représentatifs d'une distribution, avec sa diversité, et pas seulement une réponse moyenne.
- À l'entraînement, on choisit un niveau de bruit, on corrompt une donnée et on apprend à prédire le bruit ajouté.
- À la génération, on part d'un bruit aléatoire et on suit une suite de mises à jour. Le réseau et le sampler sont deux composants distincts : le réseau prédit une quantité, puis le sampler utiliser une règle pour construire l'image à partir de ces prédictions.
- Le hasard peut être présent au départ et, selon le sampler, à certaines étapes. Un sampler déterministe peut donc produire de la diversité à partir de départs différents.
- Dans la diffusion, on peut ajouter de conditions pour piloter la génération ; et on peut utiliser soit un espace latent, soit un sampler déterministe pour accélérer la génération.
- Voilà ! Pour aller plus loin : je vous ai mis des annexes après, et aussi des liens à la toute fin des slides.

Annex — One-shot MSE predicts an average



Strong noise: several clean images are plausible

An MSE predictor averages these possibilities

Take a small step, then predict again

Hugging Face Diffusion Course, Unit 1 (MNIST)

- Dans cet exemple sur les chiffres MNIST, la première ligne montre les chiffres propres. La deuxième montre leurs versions corrompues, avec de plus en plus de bruit vers la droite. La dernière ligne est la prédiction d'un réseau qui essaie de retrouver l'image propre en un seul passage.
- Quand il reste peu de bruit, la prédiction ressemble bien au chiffre initial. Quand le bruit devient très fort, plusieurs images propres peuvent expliquer l'observation : le réseau n'a plus assez d'information pour savoir laquelle était la bonne.
- Avec **une erreur quadratique moyenne**, la meilleure prédiction au sens de cet objectif est une moyenne des réponses plausibles. Si ces réponses ont des formes différentes, leur moyenne peut être floue.
- Cela ne signifie pas que tous les générateurs en un passage sont flous. Le problème est ici le **débruitage direct naïf avec une MSE**. Pour le DDPM, nous allons plutôt utiliser des prédictions dans une succession de petites mises à jour.

What should the network predict?

Noise ε	Clean image x_0	v-prediction
DDPM · SD 1.x	Denoised-image parameterization	Linear signal / noise combination Example: SD 2 at 768 px

For fixed noising, predictions can be converted into each other

Unweighted MSE losses emphasize noise levels differently

Flow-matching velocity follows a different path — slide 31

- La première colonne correspond au choix que nous suivons : prédire le **bruit** ϵ. C'est celui présenté dans DDPM et utilisé notamment par Stable Diffusion 1.x.
- On peut aussi paramétrer le modèle pour prédire une estimation de l'**image propre** x_0, ou une quantité v construite comme combinaison du signal et du bruit. La slide donne l'exemple d'une v-prediction dans Stable Diffusion 2 à 768 pixels.
- Pour un processus de bruitage fixé, ces représentations sont liées : à partir de l'état bruité et d'une prédiction, on peut convertir entre ces quantités lorsque les coefficients le permettent.
- Cela ne veut pas dire que les entraînements sont identiques : utiliser une MSE non pondérée sur une cible ou sur une autre n'accorde pas la même importance à tous les niveaux de bruit.
- **En réserve :** le v de cette paramétrisation VP n'est pas automatiquement la vitesse utilisée dans le *flow matching* que nous évoquerons en F031.
- **Transition :** « Pour la suite, on garde la prédiction du bruit ; on va regarder maintenant comment on entraîne le réseau. »

Predicting the noise and learning the score

$$s_\theta(x_t, t) := -\frac{\varepsilon_\theta(x_t, t)}{\sqrt{1-\bar{\alpha}_t}} \approx \nabla_{x_t} \log q_t(x_t)$$

$$\mu_\theta(x_t, t) = \frac{x_t + \beta_t s_\theta(x_t, t)}{\sqrt{\alpha_t}}$$

The score describes the noisy distribution at time t

Noise prediction and score estimation differ by a known scaling

Guidance changes the sampling field, not weights during inference

- Le **score** d'une distribution est le gradient de son log : intuitivement, il indique localement dans quelle direction la densité augmente. Ici, on parle de la distribution **au niveau de bruit t**, pas seulement de la distribution des images propres.
- La première formule relie la prédiction de bruit au score estimé : on prend ϵ_θ, on change son signe et on applique un facteur qui dépend du niveau de bruit. Le score du modèle approche le score de la distribution bruitée.
- La deuxième formule montre qu'on peut écrire la moyenne de la transition à partir de ce score estimé. C'est une autre manière de voir le rôle de la prédiction de bruit dans la génération.
- Cette vue est utile pour introduire le **guidage** : à l'inférence, on peut modifier les prédictions utilisées par la règle de génération pour orienter l'échantillon, sans mettre à jour les poids pendant cette génération.

In practice with diffusers

```
import torch
from diffusers import DiffusionPipeline, DPMSolverMultistepScheduler

pipe = DiffusionPipeline.from_pretrained(
    "stabilityai/stable-diffusion-xl-base-1.0",
    torch_dtype=torch.float16, variant="fp16",
).to("cuda")
pipe.scheduler = DPMSolverMultistepScheduler.from_config(
    pipe.scheduler.config, algorithm_type="dpmsolver++", solver_order=2,
)
image = pipe(
    "a photo of colorful supercomputer server racks",
    num_inference_steps=25, guidance_scale=6.0,
    generator=torch.Generator(device="cuda").manual_seed(0),
).images[0]
```

Checkpoint + compatible sampler + prompt + random seed

- La première partie du code charge un **checkpoint** préentraîné et les composants nécessaires dans un `DiffusionPipeline`, puis les place sur le GPU avec `.to("cuda")`. Si les fichiers ne sont pas déjà disponibles localement, leur récupération dépendra de l'accès au dépôt et au cache.
- On remplace ensuite le scheduler par un `DPMSolverMultistepScheduler` configuré en DPM-Solver++. Le scheduler est le composant qui applique la règle de mise à jour pendant la génération.
- Enfin, on fournit un prompt descriptif, un nombre de pas, une force de guidage et une graine aléatoire. Ces valeurs sont des **réglages d'exemple**, pas des constantes universelles ; la graine facilite les comparaisons dans un même environnement.
- Attention : le code utilise **SDXL**, alors que la slide précédente détaillait **Stable Diffusion v1**. L'organisation générale est similaire, mais les composants diffèrent ; SDXL utilise notamment deux encodeurs de texte.
- Le code est illustratif et n'a pas été exécuté ici sur GPU ; il faut vérifier les dépendances, l'accès au checkpoint et les ressources du poste utilisé.
- **Transition :** « On peut maintenant résumer le mécanisme sans entrer davantage dans le code. »

Annex — the reverse posterior, in detail

Bayes + Markov on $q(x_{t-1} \mid x_t, x_0)$ — detail of slide 16

$$q(x_{t-1} \mid x_t, x_0) = q(x_t \mid x_{t-1})\, q(x_{t-1} \mid x_0) \,/\, q(x_t \mid x_0)$$

$$q(x_{t-1} \mid x_t, x_0) = \mathcal{N}\left(x_{t-1} ; \tilde{\mu}_t(x_t, x_0), \tilde{\beta}_t \, I \right)$$
$$\tilde{\beta}_t = (1 - \bar{\alpha}_{t-1}) \,/\, (1 - \bar{\alpha}_t) \cdot \beta_t$$

$$\tilde{\mu}_t = \left(\sqrt{\bar{\alpha}_{t-1}}\, \beta_t \,/\, (1 - \bar{\alpha}_t) \right) x_0 + \left(\sqrt{\alpha_t}\, (1 - \bar{\alpha}_{t-1}) \,/\, (1 - \bar{\alpha}_t) \right) x_t$$

with $x_0 = \left(x_t - \sqrt{(1-\bar{\alpha}_t)}\, \varepsilon \right) / \sqrt{\bar{\alpha}_t}$: $\quad \tilde{\mu}_t = (1/\sqrt{\alpha_t}) \left(x_t - (\beta_t / \sqrt{(1-\bar{\alpha}_t)})\, \varepsilon \right)$

Exact given x0 · do not confuse with q(xt-1 | xt)

- Avec l’image propre connue, le posterior est gaussien et calculable.
- Bayes et la propriété de Markov donnent les coefficients affichés.
- À la génération, l’image propre manque ; le réseau fournit une estimation utile.
- Retour : « C’est ce qui motive la paramétrisation de la slide 17. »

Annex — variational bound & simplified loss

$$\mathcal{L}_{\text{VLB}} = \mathbb{E}_q\left[\log\left(\frac{q(x_{1:T} \mid x_0)}{p_\theta(x_{0:T})}\right)\right] \geq -\mathbb{E}_{q(x_0)}[\log p_\theta(x_0)]$$

$$L_{t-1} = \mathbb{E}\left[\frac{\beta_t^2}{2\sigma_t^2 \alpha_t (1-\bar{\alpha}_t)} \|\varepsilon - \varepsilon_\theta(x_t, t)\|^2\right] + C$$

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{t, x_0, \varepsilon}\left[\|\varepsilon - \varepsilon_\theta(x_t, t)\|^2\right]$$

Fixed σ_t^2, $t > 1$: denoising terms are weighted MSEs

Simple loss changes the weights and optimizes a different objective

Ho et al., 2020, sections 3.2–3.4 · https://arxiv.org/abs/2006.11239

- Objectif affiché : borne supérieure sur la log-vraisemblance négative — l'opposé de l'ELBO.
- Avec des variances fixes, les KL gaussiennes donnent des erreurs quadratiques pondérées.
- La loss simple retire ces poids et change les priorités entre niveaux de bruit.
- Meilleure qualité d'images dans DDPM ; pas nécessairement meilleure vraisemblance.
- Retour : « Voilà pourquoi une simple régression sur le bruit n'est pas un choix arbitraire. »

Annex — guidance with a differentiable loss

$$\widehat{x}_0 = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\,\varepsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}$$

$$\varepsilon_{\text{guided}} = \varepsilon_\theta + \lambda_t \sqrt{1-\bar{\alpha}_t}\,\nabla_{x_t}\ell(\widehat{x}_0)$$

Predict clean image → evaluate constraint → differentiate

Adjust noise prediction → one scheduler step

Weights frozen · strength must be tuned

Pixel-space ε model and compatible DDIM scheduler; see supplied code

Illustrative example: scripts/example_guidance.py

- Définir une contrainte différentiable sur l’image estimée.
- Calculer dans quelle direction modifier l’entrée pour réduire cette contrainte.
- Poids gelés ; gradient par rapport à l’échantillon.
- Guidage heuristique : force à régler, compromis entre contrainte et réalisme.
- Exemple en espace pixels, prédiction epsilon, DDIM sans clipping ; ce n’est pas une boucle SDXL universelle.
- Retour : « Même principe général que le classifier guidance, avec une autre fonction à favoriser. »

Annex — classifier guidance in 4 steps

1. Train the unconditional diffusion model
2. Train a classifier on noisy images x_t (it must know t)
3. Add $\lambda \cdot \nabla \log p_\varphi(y \mid x_t, t)$ to the unconditional score
4. Denoise with the guided score

Limits: a second model to train, a backward pass per step, a fixed label set, sensitivity to adversarial directions

- Un modèle génératif et un classifieur adapté aux entrées bruitées.
- À l'inférence, le gradient du classifieur oriente l'échantillon.
- Aucun réentraînement des poids pendant cette opération.
- Retour : « Le CFG évite ce classifieur auxiliaire, avec deux prédictions du débruiteur. »

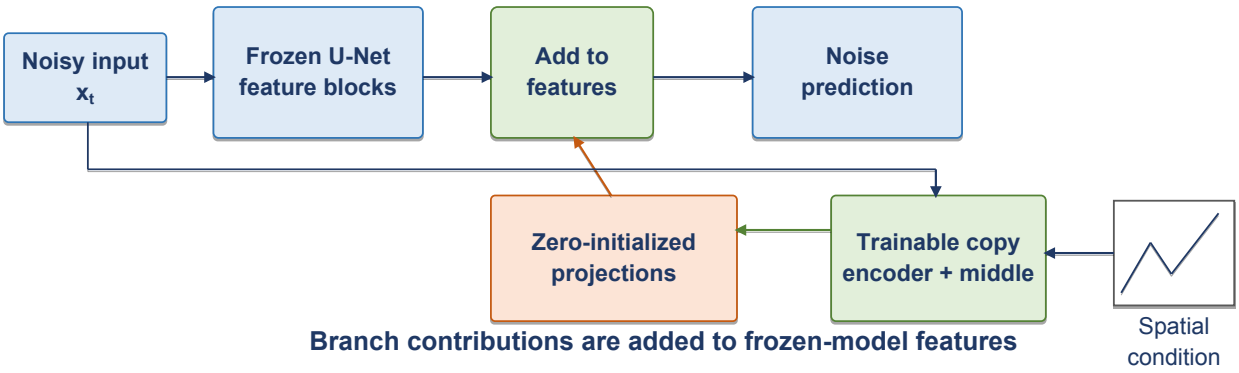
Annex — Textual Inversion & DreamBooth

Textual Inversion	DreamBooth
Learn one or a few token embeddings from reference images Keep the generative model frozen Small file tied to the text encoder and base model	Personalize a model to a specific subject Original method: fine-tuning with prior preservation Often implemented with LoRA

Gal et al., 2022 · Ruiz et al., 2022

- Textual Inversion : apprendre une représentation textuelle du concept.
- DreamBooth : adapter le modèle à un sujet particulier.
- LoRA et DreamBooth ne sont pas concurrents : l'un dit comment modifier les poids, l'autre décrit la personnalisation.
- Attention au surapprentissage et à la conservation de la variété.
- Retour : « Autre besoin : imposer une structure spatiale plutôt qu'une identité. »

Annex — ControlNet in detail



Zero-initialized projections preserve the base output at initialization

Simplified schematic adapted from Zhang et al., 2023 · arxiv.org/abs/2302.05543

- La condition spatiale entre dans la branche entraînable.
- Cette branche fournit des corrections ajoutées aux features du modèle gelé.
- Projections initialisées à zéro : aucune correction au départ.
- On apprend ensuite à utiliser les contours, la profondeur ou la pose.
- Retour : « C’est une adaptation entraînée, contrairement au guidage sans réentraînement. »

Annex — DDIM sampling rule

$$\widehat{x}_0 = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\,\varepsilon_\theta}{\sqrt{\bar{\alpha}_t}}$$

$$\sigma_{t\to s} = \eta \sqrt{\frac{1-\bar{\alpha}_s}{1-\bar{\alpha}_t}}\sqrt{1-\frac{\bar{\alpha}_t}{\bar{\alpha}_s}}$$

$$x_s = \sqrt{\bar{\alpha}_s}\,\widehat{x}_0 + \sqrt{1-\bar{\alpha}_s-\sigma_{t\to s}^2}\,\varepsilon_\theta + \sigma_{t\to s} z$$

$s < t$ · consecutive steps: $\sigma_t^2 = \eta^2 \tilde{\beta}_t$

$\eta = 0$: deterministic · same denoiser, different update

Use a coarser time grid; fewer steps trade accuracy for speed

Song et al., 2021 · https://arxiv.org/abs/2010.02502

- Estimer le signal propre, puis construire l’état au niveau de bruit suivant choisi.
- On peut sauter des niveaux tout en conservant le prédicteur.
- Sans bruit frais : déterministe pour un départ fixé.
- Moins de pas reste un compromis ; ce n’est pas simplement arrêter le DDPM plus tôt.
- Retour : « Cela explique pourquoi un même checkpoint accepte plusieurs samplers compatibles. »

Annex — masks for editing tasks

Inpainting



Black: to generate

Outpainting



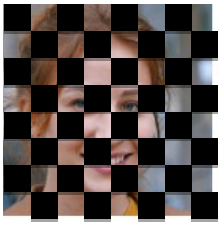
Black: to generate

Missing rows



Black: to generate

Sparse pixels



Black: to generate

Sparse-pixel reconstruction is one special case of super-resolution

Schematic masks on one source image · Lugmayr et al., RePaint, CVPR 2022

- Le masque décrit les observations disponibles.
- Même modèle préalable, différentes régions à compléter.
- Une grille de pixels observés est un cas particulier, pas toute la super-résolution.
- Retour : « Voyons comment la contrainte du masque est appliquée à chaque étape. »

Annex — RePaint: one step in detail

$$x_{t-1} = m \odot x_{t-1}^{\text{known}} + (1 - m) \odot x_{t-1}^{\text{model}}$$

m = 1: observed region · both parts are at noise level t−1

Re-noise observations, sample unknown region, merge and continue

Resampling alternates forward noising and reverse updates

Lugmayr et al., RePaint, CVPR 2022

- Bruiter l’observation au niveau de sortie de l’étape.
- Générer l’autre partie au même niveau.
- Fusionner avec le masque, puis poursuivre.
- Les allers-retours permettent d’améliorer la compatibilité entre les régions.
- Retour : « Le modèle ne change pas ; c’est la procédure de génération qui impose les observations. »

Annex — extended resources

Foundations · DDPM

Score-based SDEs

Samplers · DDIM

DPM-Solver++

Guidance · CFG

ControlNet

Latent diffusion

Diffusion Transformer (DiT)

Flow Matching

Rectified Flow

Intuition · Welch Labs / 3Blue1Brown

Practice · Hugging Face diffusion course

Click a resource to explore the topic

- Pour revoir l’intuition : Welch Labs.
- Pour implémenter : cours Hugging Face.
- Pour les détails : choisir la référence correspondant à la question, plutôt que tout lire chronologiquement.
- Fin des annexes.